When Does the GPU Pay Off? Stage-Level Costs of Stochastic Quantization in CUDA

Christian Joseph Bunyi
De La Salle University
Manila, Philippines
christian_bunyi@dlsu.edu.ph

Enzo Rafael Chan
De La Salle University
Manila, Philippines
enzo_rafael_chan@dlsu.edu.ph

Immanuel Umali
De La Salle University
Manila, Philippines
immanuel_umali@dlsu.edu.ph

Abstract

Compressing model updates before transmission reduces the bytes a training client uploads, but the compression itself costs time on the client. We propose to measure that cost for norm-scaled stochastic quantization, a QSGD-style scheme with randomized rounding that is unbiased in ideal arithmetic [1]. The proposed CUDA pipeline has three stages: a deterministic reduction for the tensor norm, counter-based stochastic rounding with the Philox generator, and packing into 4-bit or 8-bit codes behind a 20-byte header. A single-thread C implementation provides the sequential comparison, using the same reduction order, rounding arithmetic, and random words to support byte-for-byte correctness checks. The course evaluation will report stage and transfer costs, total output bytes, and the tested input sizes, if any, at which CUDA overtakes C on a consumer GPU. The project covers compression and reconstruction of synthetic tensors; federated training, model accuracy, and network transmission are outside its scope.

Keywords

Stochastic Quantization, CUDA, SIMT, Randomized Rounding, Data-Level Parallelism

1 Description of the Proposed Project

1.1 Inputs

The pipeline takes a finite FP32 vector, treated as one tensor with one scale. Each call also receives a bit width $b \in \{4, 8\}$, a 64-bit seed, a 32-bit tensor identifier, and a 32-bit invocation identifier. The seed and identifiers select the random stream by element index, so changing the CUDA launch geometry must preserve the output. Empty and all-zero tensors use a scale of zero. Nonfinite inputs and norms that cannot be represented as finite FP32 values are rejected.

The course evaluation uses dense, centered synthetic tensors at four sizes: 2^{10}, 2^{14}, 2^{18}, and 2^{22} elements, each at both bit widths. Correctness tests add edge cases: signed zero, representable extremes, values on saturation boundaries, lengths that are not multiples of the block size, and odd lengths that leave a padded nibble.

1.2 Proposed Process

1.2.1 Processes Parallelized with SIMT. The quantizer has three data-parallel stages. For bit width b, let $s = 2^{b-1} - 1$, so $s = 7$ for 4-bit codes and $s = 127$ for 8-bit codes.

Scale. The scale is the L2 norm of the tensor, $\|x\|_2$. Squaring FP32 values overflows once $|x|$ exceeds about 1.8×10^{19}, so we compute the norm with max-rescaling: $m = \max_i |x_i|$ and $\|x\|_2 = m\sqrt{\sum_i (x_i/m)^2}$. Both the maximum and the sum are reductions over all elements.

Stochastic rounding. Each element maps independently to a signed integer code. With $a_i = \min((|x_i|/\|x\|_2)\, s, s)$ and $l_i = \lfloor a_i \rfloor$, the code is $k_i = \mathrm{sign}(x_i)\,(l_i + B_i)$, where B_i is 1 with probability $a_i - l_i$ and 0 otherwise. The decoder reconstructs $\|x\|_2\, k_i/s$. Its expectation equals x_i in ideal arithmetic [1]; FP32 operations and discrete random thresholds require separate numerical checks. Each Bernoulli draw compares a 32-bit random word r with the threshold $\lfloor (a_i - l_i)\, 2^{32} \rfloor$.

Packing. Each code is stored as the unsigned value $k_i + s$, in one byte for 8-bit output or one nibble for 4-bit output. Every output byte depends on at most two codes, so all bytes can be written independently.

1.2.2 Related Implementations. Stochastic quantization of model updates is established. QSGD introduced norm-scaled stochastic quantization with bounded variance [1], FedPAQ applied quantized updates to federated learning with periodic averaging [10], and DAdaQuant adapts the quantization level over time and across clients [3]. The last two build on the communication setting of federated learning [5].

GPU implementations also exist. CGX quantizes FP16 and FP32 tensors to 1 to 8 bits in CUDA, with a stochastic mode, bit packing, and a compressed replacement for the distributed all-reduce [4]. FBGEMM passes Philox state into its GPU stochastic-rounding helpers [8], torchao documents BF16 stochastic rounding on CPU and CUDA [9], and NVIDIA provides device-side Philox generation in cuRAND [6] and hardware-accelerated stochastic rounding for NVFP4 [7]. CGX is the closest prior system, so this study does not claim a first GPU stochastic quantizer, fused quantizer, or packed format.

The proposed evaluation accounts for each stage and transfer separately, counts every header and payload byte, and fixes a counter-indexed Philox mapping [11] that makes the CPU and GPU draw the same random word for each element. These choices support byte-for-byte checks and a comparison of launch and transfer overhead against parallel computation at the selected input sizes.

1.2.3 Parallelization Plan. Figure 1 shows the proposed pipeline. The host copies the input to the device, three CUDA stages produce the packed payload, and the host copies the payload back. K1 may require several kernel launches to complete its reductions.

K1: scale. K1 runs two reduction passes: the maximum of $|x_i|$, then the sum of $(x_i/m)^2$. The initial reduction uses logical blocks of 256 elements with a tree in shared memory, following the CUDA pattern described by Harris [2]. The sum pairs adjacent values in zero-padded blocks, then pairs adjacent block sums until one remains. Grid-stride scheduling changes which CUDA block performs the work while preserving this logical tree. We avoid floating-point

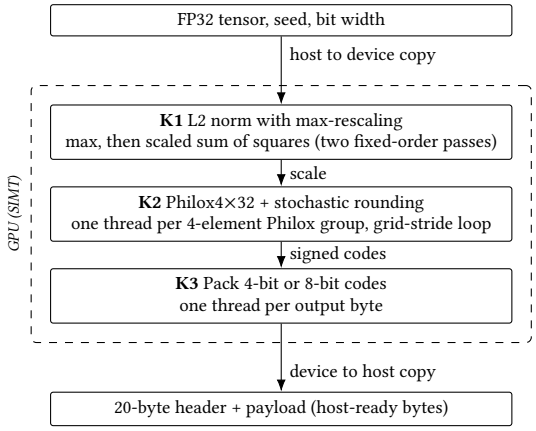


Figure 1: Proposed CUDA pipeline and the boundary between host and device.

atomics because their order of accumulation can vary between runs, which would change the scale and therefore the codes. The C baseline follows the same FP32 operations and tree; CUDA disables fused multiply-add and fast math and uses precise division and square root. Correctness checks will require identical scale bits across backends and tested grid sizes. Reading the tensor twice is a cost we will measure; fusing the passes is a later extension. We implement the reduction ourselves rather than call a library norm so that its time appears as its own stage.

K2: stochastic rounding. One call of the Philox4×32 generator with 10 rounds returns four 32-bit words [11], so each thread handles one group of four consecutive elements. The key is the seed, and the counter holds the group index $\lfloor i/4 \rfloor$, the tensor identifier, and the invocation identifier. Element i uses word i mod 4 of its group's output. Because the random word depends only on the element index, not on the thread or block that computes it, K2's launch geometry does not change the codes. Threads cover all groups with a grid-stride loop, and the final group handles the N mod 4 remaining elements. The generator is the vendored Random123 library, shared by C and CUDA. Both backends use the same division-then-multiplication order for a_i, random threshold, and packed format. Together with the fixed norm tree, these choices make byte-identical records the intended correctness criterion; prescribed scales and random words also isolate the rounding tests. Vectorized loads and other tuning are later extensions after the separate stages are correct and measured.

K3: packing. One thread writes one output byte. For 8-bit output, byte j holds $k_j + s$. For 4-bit output, byte j holds element $2j$ in the low nibble and element $2j + 1$ in the high nibble, and an odd-length tensor leaves the last high nibble zero. No two threads write the same byte, so packing needs no atomics.

1.2.4 Processes Kept Serial. The host keeps the work that is small, ordered, or outside the kernels. It generates or loads the input, rejects nonfinite values, allocates buffers, and chooses the launch configuration. It handles the zero-scale case, writes the 20-byte header, and performs the copies between host and device.

The sequential baseline is a single-thread C implementation of the same three stages. It walks the Philox groups in order, uses the same max-rescaled norm, and writes the same packed format. An FP64 Python implementation is the numerical oracle for correctness checks and stays outside the timed comparison.

1.3 Outputs

The pipeline returns host-ready bytes: a 20-byte little-endian header that records the element count and the FP32 scale, followed by a payload of $\lceil Nb/8 \rceil$ bytes. Reported compression ratios count the header bytes.

The evaluation produces three further outputs. First, correctness checks cover byte-identical C and CUDA records, isolated exact-code tests with prescribed scales and random words, reconstruction against the FP64 oracle within documented FP32 error bounds, and an empirical mean check on 1,024 elements across independent seeds. Second, we time each stage for both backends at every size and bit width, with 10 warmup runs and at least 30 measured repetitions over repeated trials, and report the median and its spread. Third, we time two boundaries: resident computation, where input and output stay in the backend's memory, and host-origin, host-ready output, where the CUDA time includes both copies. Allocation, input generation, file I/O, and correctness checks stay outside timed regions; CUDA completion is synchronized before elapsed time is recorded. Stage timings diagnose costs; the full-path comparison uses elapsed time measured over the complete boundary. The comparison identifies tested sizes at which CUDA overtakes C for each boundary and bit width, or reports that no crossover was observed in the tested range. Repeated trials must support a claimed difference; mixed results are reported as inconclusive. Locating a precise crossover through a denser size sweep is a later publication extension.

2 Target Implementation Platform

The target platform is an NVIDIA GeForce RTX 5060 with compute capability 12.0 (`sm_120`). The selected toolchain is CUDA 13.4 (`nvcc` V13.4.59), MSVC 19.51 for x64, and Random123 v1.14.0. A C host program will call CUDA launch functions through C linkage. The evaluation will record the driver and compiler versions alongside each run, and correctness checks will include the generator's known-answer vectors on CPU and GPU.

Two reproduction paths check that the build does not depend on one machine. A second team member will rebuild the code and rerun a correctness and benchmark subset on a local NVIDIA GPU, and a third will run a correctness subset on Google Colab. These runs check reproducibility and are not an additional hardware performance study.

3 Mid-Project Milestone

By Week 10, we will demonstrate the correct C pipeline and an initial CUDA scale-and-rounding path, with preliminary stage timings. The milestone is complete when the following checks pass:

- The single-thread C pipeline supports both bit widths, packing, and decoding, and passes exact-code tests for prescribed scales and random words.
- CUDA scale and rounding match the C scale bits and signed codes on a small synthetic subset at both bit widths; reconstruction passes the numerical checks against the FP64 oracle.
- A demo reconstructs CUDA-generated codes using host-side packing and decoding, and reports preliminary scale and rounding timings at 2^{10} and 2^{18} elements.

GPU packing is a stretch goal for Week 10 and is not required for this intermediate demo. The update will identify remaining work and will not treat partial-stage timings as full-pipeline speedup evidence. By Week 13, the deliverable is the complete packed CUDA pipeline, decoder and edge-case checks, the four-size timing comparison, reproducible source and run instructions, and the final demonstration.

References

[1] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. 2017. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper/2017/hash/6c340f25839e6acdc73414517203f5f0-Abstract.html

[2] Mark Harris. 2007. Optimizing Parallel Reduction in CUDA. NVIDIA Developer Technology presentation. https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf

[3] Robert Hönig, Yiren Zhao, and Robert Mullins. 2022. DAdaQuant: Doubly-Adaptive Quantization for Communication-Efficient Federated Learning. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 8852–8866. https://proceedings.mlr.press/v162/honig22a.html

[4] Ilia Markov, Hamidreza Ramezanikebrya, and Dan Alistarh. 2022. CGX: Adaptive System Support for Communication-Efficient Deep Learning. In *Proceedings of the 23rd ACM/IFIP International Middleware Conference*. ACM, 241–254. doi:10.1145/3528535.3565248

[5] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 54)*. PMLR, 1273–1282. https://proceedings.mlr.press/v54/mcmahan17a.html

[6] NVIDIA. 2026. cuRAND Device API Overview. CUDA Toolkit documentation. https://docs.nvidia.com/cuda/curand/device-api-overview.html

[7] NVIDIA. 2026. Transformer Engine: NVFP4 Low-Precision Training. Documentation. https://nvidia.github.io/TransformerEngine/features/low_precision_training/nvfp4/nvfp4.html

[8] PyTorch. 2026. FBGEMM GPU Stochastic Rounding Utilities. Source code, `fbgemm_gpu/include/fbgemm_gpu/utils/stochastic_rounding.cuh`. https://github.com/pytorch/FBGEMM/blob/main/fbgemm_gpu/include/fbgemm_gpu/utils/stochastic_rounding.cuh

[9] PyTorch. 2026. Torchao Optimizers. Documentation. https://github.com/pytorch/ao/blob/main/torchao/optim/README.md

[10] Amirhossein Reisizadeh, Aryan Mokhtari, Hamed Hassani, Ali Jadbabaie, and Ramtin Pedarsani. 2020. FedPAQ: A Communication-Efficient Federated Learning Method with Periodic Averaging and Quantization. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 108)*. PMLR, 2021–2031. https://proceedings.mlr.press/v108/reisizadeh20a.html

[11] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. 2011. Parallel Random Numbers: As Easy as 1, 2, 3. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*. ACM, 1–12. doi:10.1145/2063384.2063405